



FIAP · FASE 1

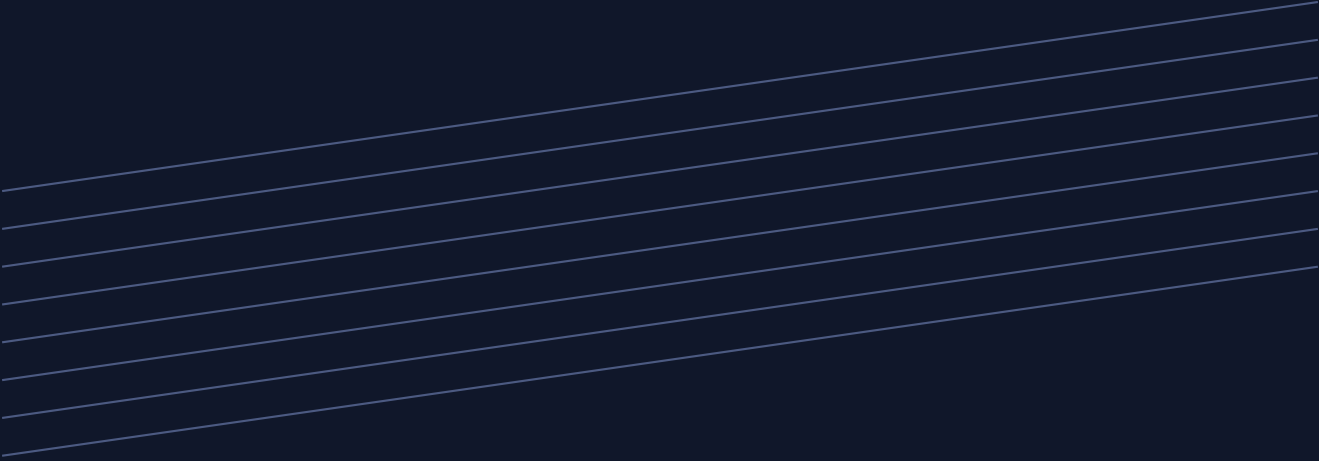
RELATÓRIO OPERACIONAL DE PRÉ-DECOLAGEM

Atividade Integradora · Decolagem da Missão

github.com/bkampdev/FIAP-PBL-FASE-1

Davi Coninck Cassaro · RM572772
Guilherme Cardoso Bremenkamp · RM574229
Lorenzo Mendes Rocha · RM575361
Gabriel Gonzales · RM574746
Eduardo Backes Klauck · RM575889

São Paulo · 16 de setembro de 2026



Resumo executivo

Este trabalho implementa uma simulação educacional de pré-decolagem. O sistema lê dados sintéticos, valida o contrato de entrada, calcula o balanço energético, verifica faixas seguras e apresenta uma decisão explicável. A missão só é liberada quando todos os critérios são satisfeitos e o saldo energético é positivo.

A análise por IA apoia a interpretação, mas não substitui as regras determinísticas nem a revisão humana. Os limites são hipóteses didáticas do grupo e não parâmetros certificados de uma nave real.

Entregáveis consolidados

Item do FIAP ON	Artefato neste repositório
1.1 Telemetria	docs/telemetria.md, dados/*.json e notebook
1.2 Algoritmo	docs/algoritmo.md e pseudocódigo
1.3 Script Python	src/, notebook e testes
1.4 Energia	src/energia.py e docs/energia.md
1.5 Análise por IA	docs/analise-ia.md
1.6 Reflexão crítica	docs/reflexao_critica.md
PDF + fonte	relatorio/relatorio-pre-decolagem.pdf e .md
README + prints	README.md e evidencias/

Links públicos

[Repositório público no GitHub](#)

[Notebook pre_decolagem.ipynb](#)

1. Objetivo e arquitetura

O objetivo é responder PRONTO PARA DECOLAR ou DECOLAGEM ABORTADA com base em telemetria e energia, acumulando os motivos encontrados.

Etapa	Responsabilidade	Módulo
1	Ler JSON e tratar arquivo ausente/malformado	src/validacao.py
2	Validar presença, tipos e domínios	src/validacao.py
3	Calcular energia inicial, perdas, saldo e autonomia	src/energia.py
4	Avaliar limites, integridade, módulos e viabilidade	src/verificacao.py
5	Integrar o fluxo e padronizar o resultado	src/missao.py
6	Apresentar texto sem recalculer valores	src/apresentacao.py

Princípios de projeto

Separação de responsabilidades. Validação de formato, cálculo energético, decisão e apresentação ficam em módulos diferentes.

Falha segura. Dado ausente, inválido ou resultado energético incompleto nunca libera a missão.

Explicabilidade. O verificador percorre todas as regras aplicáveis e acumula os motivos, em vez de ocultar problemas após a primeira falha.

Reprodutibilidade. Os cenários são JSONs versionados e o núcleo não exige API key ou acesso à internet.

2. Telemetria — requisito 1.1

O contrato distingue domínio válido de faixa operacional segura. Uma falha operacional válida deve chegar ao verificador; um valor impossível é rejeitado antes da decisão.

Campo	Unidade / tipo	Domínio válido	Faixa segura
temperatura_interna_c	°C · número	-100 a 150	15 a 30
temperatura_externa_c	°C · número	-200 a 150	-150 a 120
integridade_estrutural	inteiro 0/1	0 ou 1	1
energia_pct	% · número	0 a 100	50 a 100
pressao_tanque_kpa	kPa · número	> 0 e ≤ 1000	90 a 110
modulos	objeto	cinco módulos	todos em OK

Módulos críticos

Suporte de vida, energia, comunicação, propulsão e navegação. Os estados aceitos são OK e FALHA.

Cenários versionados

Arquivo	Alteração principal	Resultado esperado
nominal.json	nenhuma	PRONTO PARA DECOLAR
falha_temperatura.json	temperatura interna = 31 °C	aborto por temperatura
falha_modulo.json	propulsão = FALHA	aborto por módulo
falha_energia.json	consumo = 80 kWh	aborto por energia
entrada_invalida.json	pressão ausente	aborto por entrada inválida

3. Algoritmo — requisito 1.2

As fronteiras das faixas são inclusivas. A energia só é viável com saldo estritamente positivo. Todas as falhas aplicáveis são reunidas antes da decisão.

```
INÍCIO
  carregar cenário
  validar estrutura, tipos e domínios
  SE houver erros:
    retornar DECOLAGEM ABORTADA e todos os erros
  calcular energia inicial, perdas, energia útil, saldo e autonomia
  motivos <- lista vazia
  PARA cada faixa segura:
    registrar valores abaixo ou acima dos limites
  verificar integridade estrutural
  PARA cada módulo crítico:
    registrar módulo em falha
  SE saldo energético <= 0:
    registrar energia insuficiente
  SE motivos estiver vazia:
    retornar PRONTO PARA DECOLAR
  SENÃO:
    retornar DECOLAGEM ABORTADA e todos os motivos
FIM
```

Percurso nominal

Temperaturas, energia e pressão estão dentro das faixas; a integridade é 1; todos os módulos estão em OK; o saldo energético é 56 kWh. Como a lista de motivos permanece vazia, o resultado é PRONTO PARA DECOLAR.

Percurso com falha

Com temperatura interna de 31 °C, o verificador registra que o valor excede o máximo de 30 °C. O saldo energético positivo não elimina a falha térmica. A decisão é DECOLAGEM ABORTADA e o motivo permanece visível.

Contrato de saída

```
{"decisao": "PRONTO PARA DECOLAR", "motivos": []}
{"decisao": "DECOLAGEM ABORTADA", "motivos": ["..."]}
```

4. Script Python — requisito 1.3

O notebook importa os módulos do repositório e executa cenários reais. O trecho abaixo resume a integração; cada função mantém uma responsabilidade única.

```
from src.missao import executar_cenario, LIMITES_PADRAO
from src.apresentacao import formatar_resultado

# Executar a partir da raiz do repositório.
resultado = executar_cenario("dados/nominal.json", LIMITES_PADRAO)
print(formatar_resultado(resultado))
```

Tratamento de erros

Arquivo ausente, JSON malformatado, campo obrigatório ausente, tipo incorreto, número não finito ou percentuais fora de domínio geram aborto explicado. Nenhum dado crítico é preenchido silenciosamente com um valor seguro.

Apresentação

A função `formatar_resultado` recebe o dicionário final e retorna uma string. Ela mostra cenário, decisão, motivos e energia, lida com valores indisponíveis e não altera o objeto recebido.

Testes automatizados

A suíte cobre nominal, limites, cada módulo crítico, integridade, energia positiva/nula/negativa, entradas inválidas, integração e apresentação. Na versão final, 52 testes foram aprovados.

5. Análise energética — requisito 1.4

A capacidade total, a carga, as perdas e o consumo são expressos em kWh/%; a potência média é usada somente para estimar a autonomia após a decolagem.

Etapa	Fórmula	Exemplo nominal
Energia inicial	$\text{capacidade} \times \text{carga} / 100$	$100 \times 80\% = 80 \text{ kWh}$
Perdas	$\text{energia inicial} \times \text{perdas} / 100$	$80 \times 5\% = 4 \text{ kWh}$
Energia útil	$\text{energia inicial} - \text{perdas}$	$80 - 4 = 76 \text{ kWh}$
Saldo	$\text{energia útil} - \text{consumo}$	$76 - 20 = 56 \text{ kWh}$
Autonomia	$\text{saldo} / \text{potência, se saldo} > 0$	$56 / 10 = 5,6 \text{ h}$

Regra de viabilidade: saldo maior que zero. Com saldo zero ou negativo, viável é falso e a autonomia é None.

Cenário de falha energética

Com a mesma energia inicial e perdas, mas consumo de decolagem de 80 kWh, o saldo é $76 - 80 = -4 \text{ kWh}$. A decisão é DECOLAGEM ABORTADA e a autonomia temporal não é calculada.

Validações numéricas

Capacidade precisa ser positiva; carga e perdas devem estar entre 0% e 100%; consumo não pode ser negativo; potência, quando informada, precisa ser positiva. NaN e infinito são rejeitados.

6. Análise assistida por IA — requisito 1.5

A consulta ao Codex/GPT-5 pediu classificação dos dados, anomalias e sugestões de risco para os cenários nominal, temperatura e propulsão. O prompt forneceu as faixas e fórmulas e proibiu autorização operacional real.

Cenário	Classificação da IA	Revisão humana
Nominal	sem anomalias; saldo 56 kWh; autonomia 5,6 h	confere com JSON e código
Temperatura	31 °C excede o máximo de 30 °C	confere; energia não elimina a falha
Propulsão	módulo crítico em FALHA	confere; exige aborto
Limitação	não há dados para causa interna da falha	correto; não inventar diagnóstico

Em caso de divergência, prevalecem os dados versionados e os módulos determinísticos de validação, energia e verificação.

Geração sintética opcional

O módulo `src/geracao.py` usa `GaussianMixture` como extensão para produzir dados sintéticos. Ele não participa da decisão autoritativa e não é necessário para a execução do notebook principal.

7. Reflexão crítica — requisito 1.6

Ética e responsabilidade

Uma decisão de segurança precisa ser explicável. O sistema acumula motivos, informa valores e limites, e aborta quando faltam dados indispensáveis. A IA serve como apoio interpretativo; não recebe autoridade para aprovar a missão.

Impacto social

Tecnologias espaciais podem gerar aplicações em medicina, agricultura, manufatura e observação terrestre. Ao mesmo tempo, o investimento espacial disputa recursos com outras prioridades sociais. O simulador não resolve essa decisão política, mas evita apresentar dados sintéticos como segurança real.

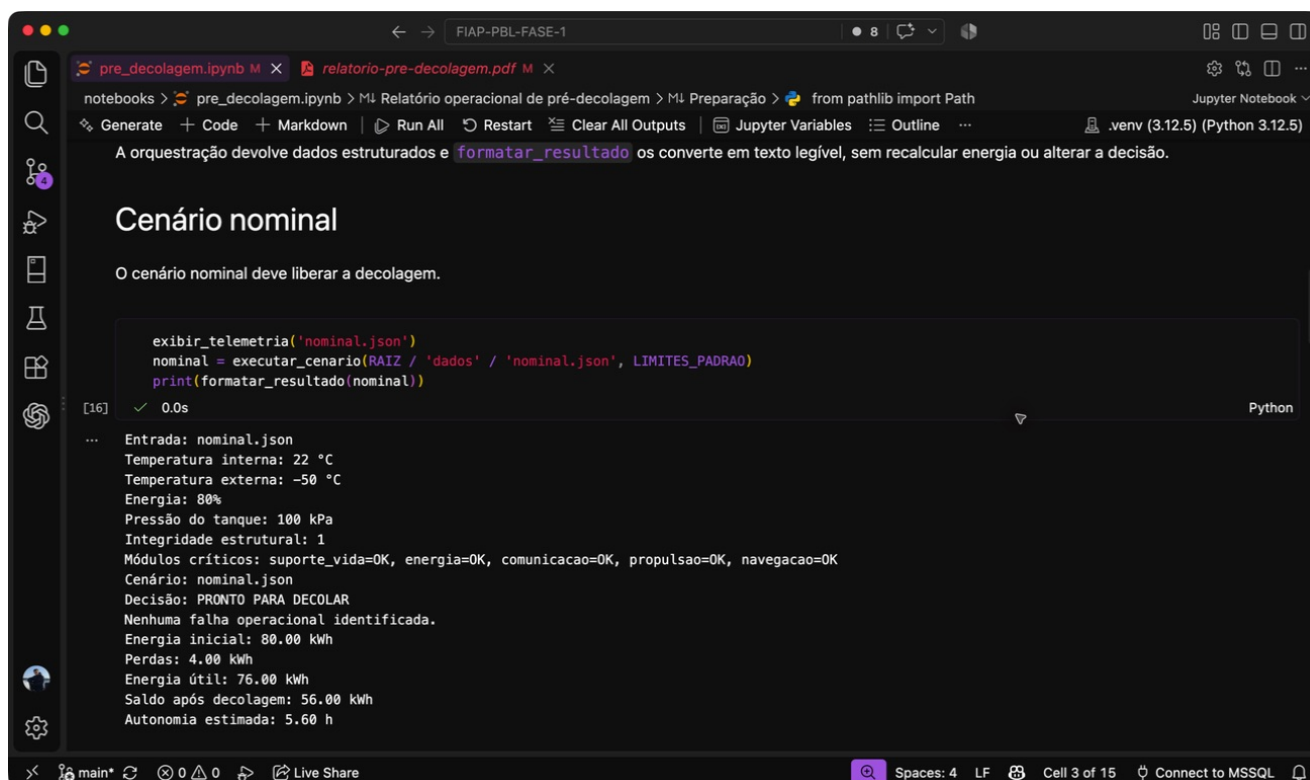
Sustentabilidade tecnológica

O balanço energético torna perdas e consumo explícitos e evita declarar autonomia quando o saldo é nulo ou negativo. O projeto não mediu emissões ou consumo computacional; portanto, não faz alegações ambientais sem dados.

Síntese

O principal aprendizado é que uma verificação segura não se limita a responder “sim” ou “não”: ela deve explicar, registrar incertezas e permitir auditoria.

8. Evidências — cenário nominal



```
from pathlib import Path

A orquestração devolve dados estruturados e formatar_resultado os converte em texto legível, sem recalculer energia ou alterar a decisão.
```

Cenário nominal

O cenário nominal deve liberar a decolagem.

```
exibir_telemetria('nominal.json')
nominal = executar_cenario(RAIZ / 'dados' / 'nominal.json', LIMITES_PADRAO)
print(formatar_resultado(nominal))
```

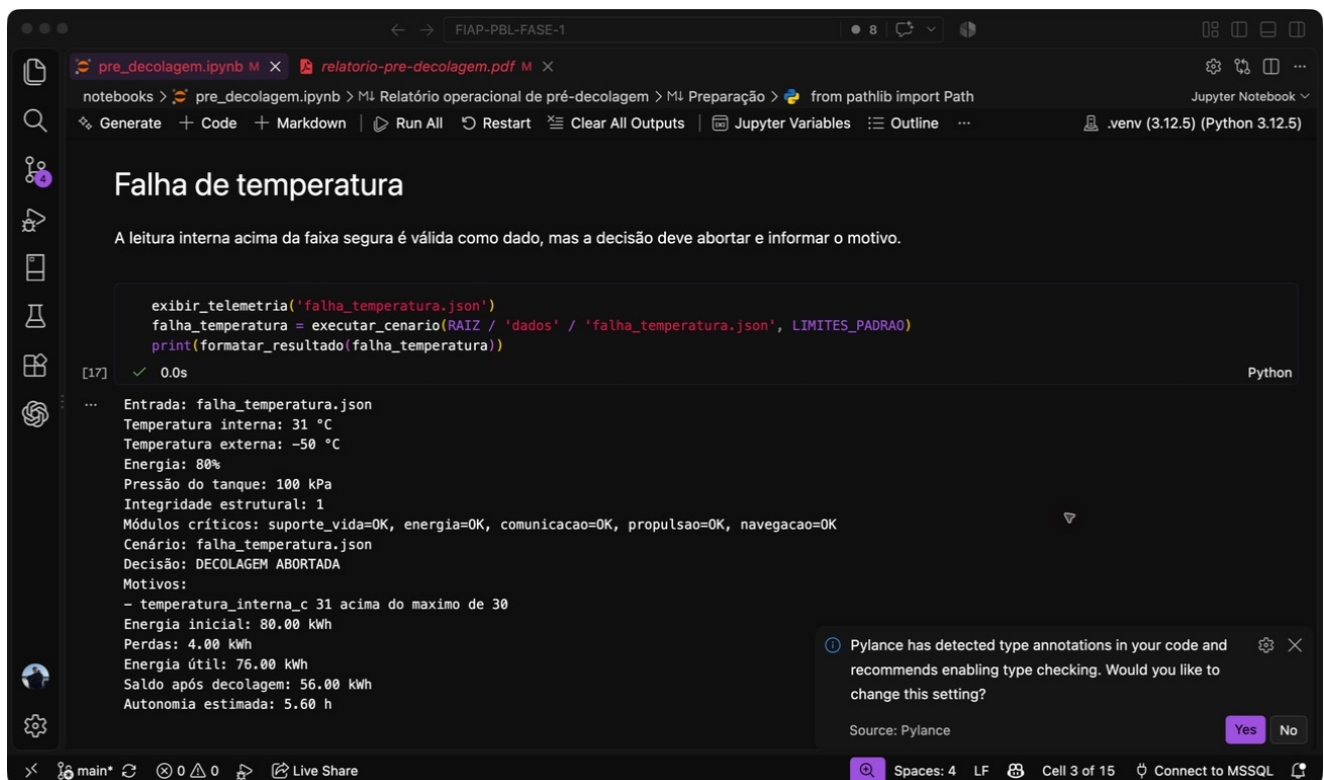
[16] ✓ 0.0s Python

Entrada: nominal.json
Temperatura interna: 22 °C
Temperatura externa: -50 °C
Energia: 80%
Pressão do tanque: 100 kPa
Integridade estrutural: 1
Módulos críticos: suporte_vida=OK, energia=OK, comunicacao=OK, propulsao=OK, navegacao=OK
Cenário: nominal.json
Decisão: PRONTO PARA DECOLAR
Nenhuma falha operacional identificada.
Energia inicial: 80.00 kWh
Perdas: 4.00 kWh
Energia útil: 76.00 kWh
Saldo após decolagem: 56.00 kWh
Autonomia estimada: 5.60 h

Figura 1 — Captura real do VS Code: telemetria nominal, saldo de 56 kWh, autonomia de 5,6 h e decisão PRONTO PARA DECOLAR.

Captura real da janela do VS Code após executar o notebook com o kernel Python 3.12.5. A imagem preserva código, contador e saída da execução. O JSON de origem é dados/nominal.json.

9. Evidências — aborto por temperatura



The screenshot shows a VS Code window with a Jupyter Notebook titled 'pre_decolagem.ipynb'. The notebook is open to a cell containing Python code that simulates a temperature failure scenario. The code reads a JSON file 'falha_temperatura.json' and executes a scenario using 'RAIZ' and 'LIMITES_PADRAO'. The output of the cell shows the following data:

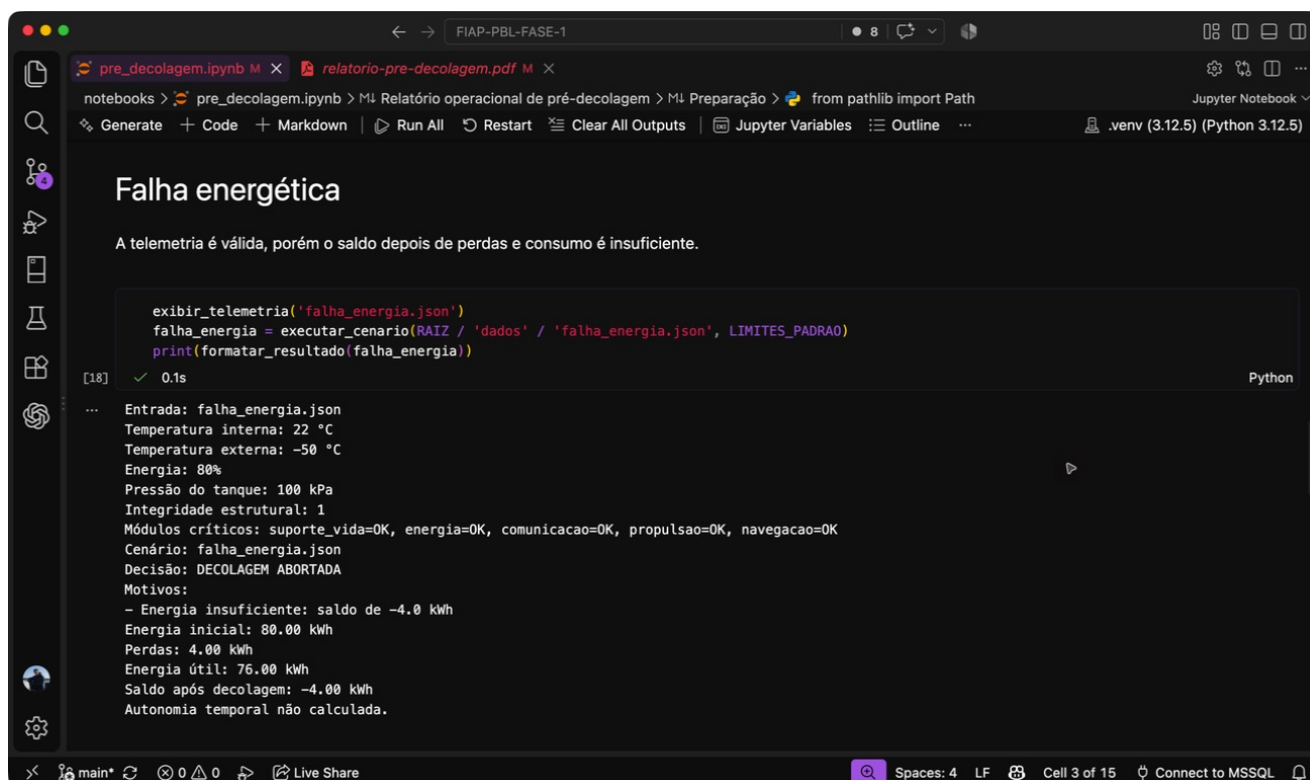
```
Entrada: falha_temperatura.json
Temperatura interna: 31 °C
Temperatura externa: -50 °C
Energia: 80%
Pressão do tanque: 100 kPa
Integridade estrutural: 1
Módulos críticos: suporte_vida=OK, energia=OK, comunicacao=OK, propulsao=OK, navegacao=OK
Cenário: falha_temperatura.json
Decisão: DECOLAGEM ABORTADA
Motivos:
- temperatura_interna_c 31 acima do maximo de 30
Energia inicial: 80.00 kWh
Perdas: 4.00 kWh
Energia útil: 76.00 kWh
Saldo após decolagem: 56.00 kWh
Autonomia estimada: 5.60 h
```

A Pylance notification is visible in the bottom right corner, stating: "Pylance has detected type annotations in your code and recommends enabling type checking. Would you like to change this setting?" with "Yes" and "No" buttons.

Figura 2 — Captura real do VS Code: temperatura interna de 31 °C, acima do limite de 30 °C, e decisão DECOLAGEM ABORTADA.

Os demais dados permanecem nominais, o que isola a causa do aborto. O JSON de origem é dados/falha_temperatura.json.

10. Evidências — aborto por energia



```
from pathlib import Path

exibir_telemetria('falha_energia.json')
falha_energia = executar_cenario(RAIZ / 'dados' / 'falha_energia.json', LIMITES_PADRAO)
print(formatar_resultado(falha_energia))
```

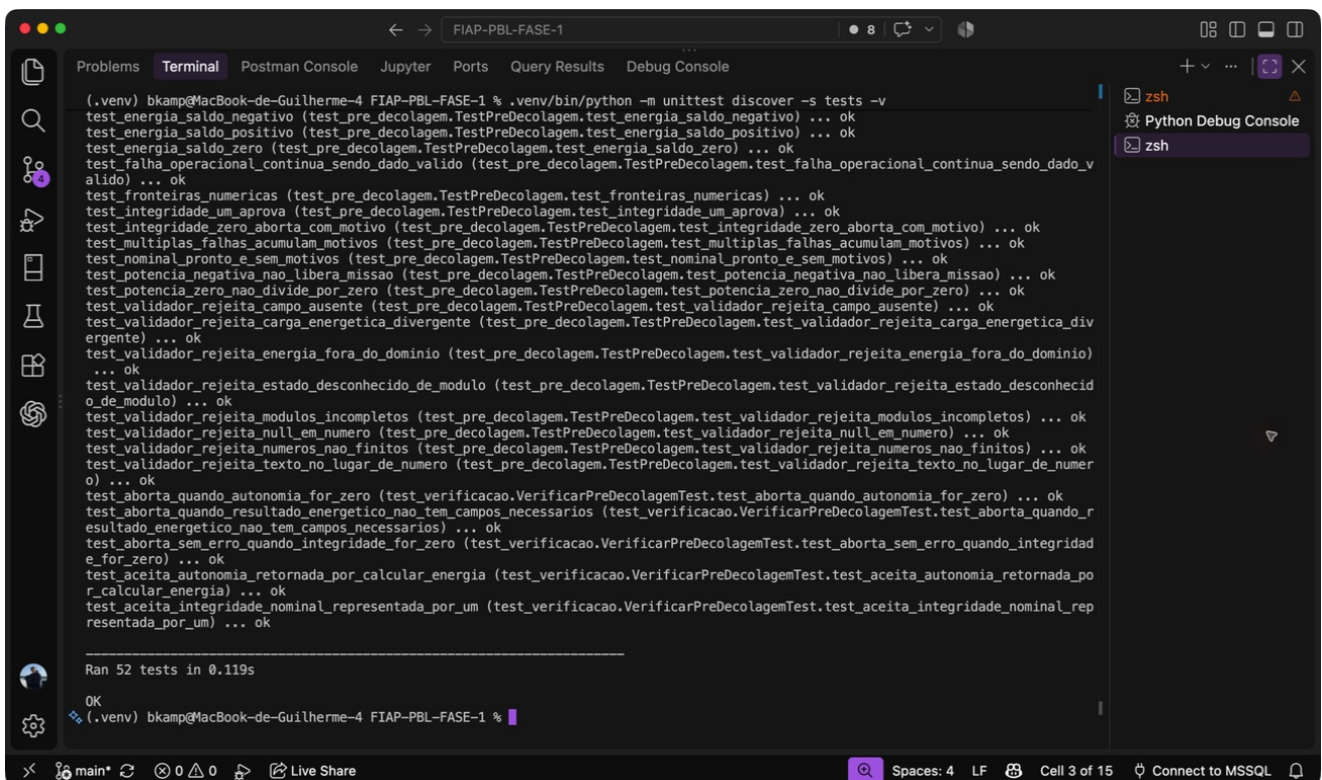
[18] ✓ 0.1s Python

Entrada: falha_energia.json
Temperatura interna: 22 °C
Temperatura externa: -50 °C
Energia: 80%
Pressão do tanque: 100 kPa
Integridade estrutural: 1
Módulos críticos: suporte_vida=OK, energia=OK, comunicacao=OK, propulsao=OK, navegacao=OK
Cenário: falha_energia.json
Decisão: DECOLAGEM ABORTADA
Motivos:
- Energia insuficiente: saldo de -4.0 kWh
Energia inicial: 80.00 kWh
Perdas: 4.00 kWh
Energia útil: 76.00 kWh
Saldo após decolagem: -4.00 kWh
Autonomia temporal não calculada.

Figura 3 — Captura real do VS Code: saldo de -4 kWh, autonomia não calculada e decisão DECOLAGEM ABORTADA.

O consumo de decolagem de 80 kWh supera a energia útil de 76 kWh. O JSON de origem é dados/falha_energia.json.

11. Evidências — testes no VS Code



```
(.venv) bkamp@MacBook-de-Guilherme-4 FIAP-PBL-FASE-1 % .venv/bin/python -m unittest discover -s tests -v
test_energia_saldo_negativo (test_pre_decolagem.TestPreDecolagem.test_energia_saldo_negativo) ... ok
test_energia_saldo_positivo (test_pre_decolagem.TestPreDecolagem.test_energia_saldo_positivo) ... ok
test_energia_saldo_zero (test_pre_decolagem.TestPreDecolagem.test_energia_saldo_zero) ... ok
test_falha_operacional_continua_sendo_dado_valido (test_pre_decolagem.TestPreDecolagem.test_falha_operacional_continua_sendo_dado_v
alido) ... ok
test_fronteras_numericas (test_pre_decolagem.TestPreDecolagem.test_fronteras_numericas) ... ok
test_integridade_um_aprova (test_pre_decolagem.TestPreDecolagem.test_integridade_um_aprova) ... ok
test_integridade_zero_aborta_com_motivo (test_pre_decolagem.TestPreDecolagem.test_integridade_zero_aborta_com_motivo) ... ok
test_multiplas_falhas_acumulam_motivos (test_pre_decolagem.TestPreDecolagem.test_multiplas_falhas_acumulam_motivos) ... ok
test_nominal_pronto_e_sem_motivos (test_pre_decolagem.TestPreDecolagem.test_nominal_pronto_e_sem_motivos) ... ok
test_potencia_negativa_nao_libera_missao (test_pre_decolagem.TestPreDecolagem.test_potencia_negativa_nao_libera_missao) ... ok
test_potencia_zero_nao_divide_por_zero (test_pre_decolagem.TestPreDecolagem.test_potencia_zero_nao_divide_por_zero) ... ok
test_validador_rejeita_campo_ausente (test_pre_decolagem.TestPreDecolagem.test_validador_rejeita_campo_ausente) ... ok
test_validador_rejeita_carga_energetica_divergente (test_pre_decolagem.TestPreDecolagem.test_validador_rejeita_carga_energetica_div
ergente) ... ok
test_validador_rejeita_energia_fora_do dominio (test_pre_decolagem.TestPreDecolagem.test_validador_rejeita_energia_fora_do dominio)
... ok
test_validador_rejeita_estado_desconhecido_de_modulo (test_pre_decolagem.TestPreDecolagem.test_validador_rejeita_estado_desconhecid
o_de_modulo) ... ok
test_validador_rejeita_modulos_incompletos (test_pre_decolagem.TestPreDecolagem.test_validador_rejeita_modulos_incompletos) ... ok
test_validador_rejeita_null_em_numero (test_pre_decolagem.TestPreDecolagem.test_validador_rejeita_null_em_numero) ... ok
test_validador_rejeita_numeros_nao_finitos (test_pre_decolagem.TestPreDecolagem.test_validador_rejeita_numeros_nao_finitos) ... ok
test_validador_rejeita_texto_no_lugar_de_numero (test_pre_decolagem.TestPreDecolagem.test_validador_rejeita_texto_no_lugar_de_numer
o) ... ok
test_aborta_quando_autonomia_for_zero (test_verificacao.VerificarPreDecolagemTest.test_aborta_quando_autonomia_for_zero) ... ok
test_aborta_quando_resultado_energetico_nao_tem_campos_necessarios (test_verificacao.VerificarPreDecolagemTest.test_aborta_quando_r
esultado_energetico_nao_tem_campos_necessarios) ... ok
test_aborta_sem_erro_quando_integridade_for_zero (test_verificacao.VerificarPreDecolagemTest.test_aborta_sem_erro_quando_integridad
e_for_zero) ... ok
test_aceita_autonomia_retornada_por_calcular_energia (test_verificacao.VerificarPreDecolagemTest.test_aceita_autonomia_retornada_po
r_calcular_energia) ... ok
test_aceita_integridade_nominal_representada_por_um (test_verificacao.VerificarPreDecolagemTest.test_aceita_integridade_nominal_rep
resentada_por_um) ... ok

Ran 52 tests in 0.119s

OK
(.venv) bkamp@MacBook-de-Guilherme-4 FIAP-PBL-FASE-1 %
```

Figura 4 — Terminal real do VS Code: 52 testes executados e resultado OK.

Comando executado no terminal integrado: `.venv/bin/python -m unittest discover -s tests -v`. A captura mostra o fim da execução, a contagem e o resultado, sem recriação da interface.

12. Gerador de telemetria integrado

O notebook importa gerar_cenario de src.geracao, executar_cenario de src.missao e a apresentação de src.apresentacao. A saída é textual e mostra temperaturas, integridade, energia, pressão, módulos, decisão, motivos e autonomia.

Correção da base sintética

A base de treinamento continha pressões próximas de 420 kPa, incompatíveis com a faixa didática de segurança de 90 a 110 kPa. A base sintética foi corrigida para aproximadamente 94 a 104 kPa, centrada perto de 100 kPa. Não houve conversão de unidades nem alteração dos limites do verificador.

Aleatoriedade e decisão

Cada chamada sem seed fixa sorteia novos dados pelo modelo GaussianMixture e pelos estados dos módulos. As amostras são preservadas, sem substituição por JSON fixo ou ajuste posterior para garantir aprovação. O cenário nominal não força falhas, mas ainda pode abortar por falhas sorteadas nos módulos.

Os cenários energia_insuficiente, falha_modulo e falha_sensor introduzem falhas deliberadas e devem resultar em DECOLAGEM ABORTADA.

Verificação da correção

Em uma execução de verificação com 100 sorteios nominais sem seed fixa, ocorreram 87 aprovações e 13 abortos. Esta contagem descreve apenas a execução observada; não é uma taxa garantida. Os 52 testes passaram e o notebook foi executado integralmente após a correção.

As capturas anteriores são execuções reais dos cenários JSON determinísticos. Elas não representam esta amostragem aleatória. A análise assistida por IA do item 1.5 permanece documentada separadamente do gerador.

13. Validação, conclusão e referências

Verificação executada em 16/09/2026

```
python -m unittest discover -s tests -v
python -m jupyter nbconvert --to notebook --execute --inplace notebooks/pre_decolagem.ipynb
```

Resultado: 52 testes aprovados, 0 falhas, e notebook executado integralmente.

Cenário	Decisão	Saldo	Autonomia
nominal	PRONTO PARA DECOLAR	56 kWh	5,6 h
falha de temperatura	DECOLAGEM ABORTADA	56 kWh	5,6 h
falha de energia	DECOLAGEM ABORTADA	-4 kWh	não calculada
entrada inválida	DECOLAGEM ABORTADA	indisponível	indisponível

Conclusão

O projeto atende aos seis itens técnicos solicitados e inclui notebook executável, README com instruções e prints, fonte editável e PDF. A separação entre validação, energia, decisão e apresentação permite rastrear cada resultado e mantém explícito o caráter didático dos dados.

Referências

FIAP ON. Atividade Integradora — Relatório Operacional de Pré-Decolagem. Consulta em 16 set. 2026.

NASA. Technology Transfer and Spinoffs. nasa.gov.

Projeto público: github.com/bkampdev/FIAP-PBL-FASE-1.

Base integrada: a487de4. O notebook e os artefatos foram consolidados em 16/09/2026. A presença do PDF no GitHub não equivale à submissão no FIAP ON.